Notes 9

1. `grep`

DEFINITION:

- `grep` searches for patterns (text) inside files and prints matching lines. Basically, find lines that contain this.

USAGE/FORMULA:

- `grep` "pattern" [file_name]
- `grep` -i "pattern" [file_name] (Ignore case)
- `grep` -n "pattern" [file_name] (Show line numbers)
- `grep` -v "pattern" [file_name] (Show lines that DO NOT match)
- `grep` -c "pattern" [file_name] (Count matches)

EXAMPLES:

- `grep` "error" access.log (Search for error)
- `grep` -i "error" access.log (Case-sensitive search)
- `grep` -n "error" access.log (Show line numbers)
- `grep` -v "error" access.log (Show lines that do NOT contain “error”)

2. `awk`

DEFINITION:

- `awk` is a text processing tool used to analyze and manipulate structured data (like columns).

USAGE/FORMULA:

- `awk` 'pattern { action }' [file_name]
- `awk` -F "delimiter" '{print $column}' [file_name]

NOTE: It doesn't actually use a bracket for file name, I just did that to separate the syntax a bit.

REMEMBER THIS:

- $1 = first column
- $2 = second column
- $0 = entire line
- -F = field separator (delimiter)

EXAMPLES:

- `awk` -F "," '{print $1}' students.csv (Print first column; comma separated file)
- `awk` -F "," '{print $1, $3}' students.csv (Print first and third columns)
- `awk` '{print $0}' file.txt (Print entire line)

3. `sed`

DEFINITION:

- `sed` (stream editor) is used to modify text automatically, especially for replacing text. Think: “Find and replace in a file.”

USAGE/FORMULA:

- `sed` 's/old/new/' [file_name]

THE SYNTAX:

- s = substitute
- s/old/new/ = replace first match per line
- s/old/new/g = replace ALL matches per line

EXAMPLES:

- `sed` 's/cat/dog/' file.txt (Replace “cat” with “dog”; first occurrence per line)
- `sed` 's/cat/dog/g' file.txt (Replace all occurrences)
- `sed` 's/;/,/g' file.csv (Replace delimiter)
- `sed` '/error/s/fail/success/' file.txt (Replace only in lines containing a word)

4. How/When to use pipe | (redirecting the output of a command to another):

Use a pipe when:

- You want to process data step-by-step
- You don’t need to save the result, just refine it
- You want to chain commands together

3 examples on how to use it:

- cat access.log | grep "error"
- cut -d "," -f 1 students.csv | sort
- sort names.txt | wc -l

5. How/When to use > (save the output of a command to a file):

Use > when:

- You want to store results
- You need a file for later use
- You are creating output files for assignments

3 examples on how to use it:

- grep "error" access.log > errors.txt
- sort names.txt > sorted_names.txt
- wc -l access.log > count.txt

How/When to use append >> (append the output of a command to a file):

Use append >> when:

- You want to add more data to an existing file
- You are building a log or report over time
- You don’t want to overwrite previous results

3 examples on how to use it:

- grep "error" access.log >> errors.txt
- date >> log.txt
- wc -l access.log >> report.txt